

Extended Kalman Filter and Localization Techniques

Pravin Oli (u1999699), and Gebrecherkos Gebreslassie (u1999542)

December 29, 2024

Abstract

This report presents the implementation and evaluation of map-based Extended Kalman Filter (EKF) localization techniques for differential drive mobile robots. Various scenarios, including input displacement and constant velocity motion models with 2D Cartesian and polar feature observations, were tested under different update frequencies for compass and feature measurements. The study explores the impact of observation frequencies on localization accuracy, highlighting scenarios with and without feature updates. Challenges such as plotting errors in polar observations were identified. The results demonstrate the importance of frequent updates in reducing drift and improving state estimation accuracy.

Contents

1	Introduction	3
1.1	Gaussian Filter (with Multivariate Gaussian)	3
1.2	Kalman Filter	3
1.2.1	State Transition Model	3
1.2.2	Measurement Model	4
1.2.3	Steps of the Kalman Filter	4
1.3	Extended Kalman Filter (EKF)	4
1.3.1	Nonlinear Models	4
1.3.2	Steps of the EKF	4
1.4	Map-Based Localization	5
1.4.1	Bayesian Formulation	5
2	Methodology	5
3	Implementation	6
4	Result and Discussion	9
5	Problems	15
6	Conclusion	16

1 Introduction

State estimation and localization essential in robotics, to determine their position and navigate autonomous system accurately. This lab focus was on solving the localization problem of a Differential Drive Mobile Robot using a pre-known map of point features. These features were stored in Cartesian coordinates but observed by the robot in polar coordinates. The lab emphasized implementing a map-based Extended Kalman Filter (EKF) for localization. The EKF, an extension of the Kalman Filter, effectively handles non-linear motion and measurement models by linearizing them using Jacobian matrices. The lab incorporated a 3DOF differential drive motion model, constant velocity assumptions, and transformations between Cartesian and polar coordinates.

This section covers foundational concepts on Gaussian filtering techniques, Kalman filtering, and their extensions for nonlinear systems.

1.1 Gaussian Filter (with Multivariate Gaussian)

The Gaussian filter assumes that the state $\mathbf{x}$ follows a multivariate Gaussian distribution:

$$p(\mathbf{x}) \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

Where:

- $\boldsymbol{\mu}$: Mean vector (state estimate).
- $\boldsymbol{\Sigma}$: Covariance matrix (uncertainty of the state).

The probability density function (PDF) of the multivariate Gaussian is given by:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{n/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right)$$

Where:

- n : Dimension of the state $\mathbf{x}$.
- $|\boldsymbol{\Sigma}|$: Determinant of the covariance matrix.
- $\boldsymbol{\Sigma}^{-1}$: Inverse of the covariance matrix.

1.2 Kalman Filter

The Kalman filter is a recursive estimator for linear systems. It consists of the following models:

1.2.1 State Transition Model

$$\mathbf{x}_k = \mathbf{A}\mathbf{x}_{k-1} + \mathbf{B}\mathbf{u}_k + \mathbf{w}_k$$

1.2.2 Measurement Model

$$\mathbf{z}_k = \mathbf{H}\mathbf{x}_k + \mathbf{v}_k$$

Where:

- $\mathbf{x}_k$: State at time k .
- $\mathbf{u}_k$: Control input.
- $\mathbf{z}_k$: Measurement at time k .
- $\mathbf{A}, \mathbf{B}, \mathbf{H}$: State transition, control, and measurement matrices.
- $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q})$: Process noise.
- $\mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R})$: Measurement noise.

1.2.3 Steps of the Kalman Filter

Prediction:

$$\begin{aligned}\hat{\mathbf{x}}_k^- &= \mathbf{A}\hat{\mathbf{x}}_{k-1} + \mathbf{B}\mathbf{u}_k \\ \mathbf{P}_k^- &= \mathbf{A}\mathbf{P}_{k-1}\mathbf{A}^\top + \mathbf{Q}\end{aligned}$$

Update:

$$\begin{aligned}\mathbf{K}_k &= \mathbf{P}_k^- \mathbf{H}^\top (\mathbf{H}\mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R})^{-1} \\ \hat{\mathbf{x}}_k &= \hat{\mathbf{x}}_k^- + \mathbf{K}_k(\mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_k^-) \\ \mathbf{P}_k &= (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_k^-\end{aligned}$$

1.3 Extended Kalman Filter (EKF)

The EKF extends the Kalman filter to handle nonlinear systems using linearization.

1.3.1 Nonlinear Models

$$\begin{aligned}\mathbf{x}_k &= f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k \\ \mathbf{z}_k &= h(\mathbf{x}_k) + \mathbf{v}_k\end{aligned}$$

1.3.2 Steps of the EKF

Linearization using Jacobians:

$$\mathbf{F}_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k}, \quad \mathbf{H}_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k^-}$$

Prediction:

$$\begin{aligned}\hat{\mathbf{x}}_k^- &= f(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k) \\ \mathbf{P}_k^- &= \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}\end{aligned}$$

Update:

$$\begin{aligned}\mathbf{K}_k &= \mathbf{P}_k^- \mathbf{H}_k^\top (\mathbf{H}_k \mathbf{P}_k^- \mathbf{H}_k^\top + \mathbf{R})^{-1} \\ \hat{\mathbf{x}}_k &= \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_k^-)) \\ \mathbf{P}_k &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_k^-\end{aligned}$$

1.4 Map-Based Localization

Map-based localization estimates the robot's pose $\mathbf{x}_k$ relative to a known map M using sensor data $\mathbf{z}_k$.

1.4.1 Bayesian Formulation

$$p(\mathbf{x}_k | \mathbf{z}_{1:k}, \mathbf{u}_{1:k}, M) \propto p(\mathbf{z}_k | \mathbf{x}_k, M) p(\mathbf{x}_k | \mathbf{u}_k, \mathbf{x}_{k-1})$$

Where:

- $p(\mathbf{x}_k | \mathbf{u}_k, \mathbf{x}_{k-1})$: Motion model.
- $p(\mathbf{z}_k | \mathbf{x}_k, M)$: Sensor model.

The motion model predicts the robot's position based on its movement, while the sensor model aligns observations with the known map.

2 Methodology

This section describes the overall stepwise approach used in implementing the lab.

- **Step 1: Cloning the PR.lab4 Repository:** Clone the repository to initialize the lab environment.
- **Step 2: Updating with Lab 1 Solutions:**
 - Updated DifferentialDriveSimulatedRobot.py and DR_3DOFDifferentialDrive.py.
 - Replaced Pose3D.py with Pose.py and updated imports accordingly.
- **Step 3: Localization using Dead Reckoning and a Compass:**
 - Completed missing code in GFLocalization, EKF, EKF_3DOFDifferentialDrive InputDisplacement classes, and EKF.py.
- **Step 4: Cartesian Feature Implementation:**

- Completed missing methods (boxplus, j1_boxplus, and j2_boxplus) in the CartesianFeature class in Feature.py.

- **Step 5: Map Feature Implementation:**

- Completed missing methods in the MapFeature and Cartesian2DMapFeature classes in MapFeature.py.

- **Step 6: Map-Based EKF Localization:**

- Completed get_measurement, get_feature_extraction, innovation, uncertainty, and Mahalanobis distance calculations in the FKEKFMBL class in FKEKFMBL.py.
- Tested under the following conditions:
 - * Condition 1: Without compass updates and without feature updates. Adjusted yaw_reading_frequency and xy_feature_reading_frequency to achieve this.
 - * Condition 2: Without compass updates but with feature updates every 50 simulation steps. Adjusted yaw_reading_frequency and xy_feature_reading_frequency accordingly.

3 Implementation

The implementation of this lab is done on three parts.

Part 1 focuses on implementation of an Extended Kalman Filter (EKF) Localization method using the robot odometry displacement as the motion model and making updates with the compass readings (constant velocity model is optional).

Part 2 focuses on implementation of feature class transforming from polar to Cartesian, from spherical to Cartesian, and vice versa, each accompanied by their respective Jacobians.

Part 3 for incorporation of map features in robot localization. Within this class, we have implemented methods such as S2O for converting from storage to observation and O2S, which employs a conversion function from the observation representation to the storage representation. Additionally, we have implemented their respective Jacobians, J_s2o and J_o2s.

Displacement Model

State Vector

$$\mathbf{x}_k = \begin{bmatrix} N_x^B \\ N_y^B \\ N_\theta^B \end{bmatrix} \sim \mathcal{N}(\hat{\mathbf{x}}_k, \mathbf{P}_k)$$

Motion Model

$$\begin{aligned} \bar{\mathbf{x}}_k &= \hat{\mathbf{x}}_{k-1} \oplus \mathbf{u}_k + w_k \\ \bar{\mathbf{P}}_k &= A_k \mathbf{P}_{k-1} A_k^T + W_k \mathbf{Q}_k W_k^T \end{aligned}$$

Jacobian with Respect to State Vector

$$A_k = \begin{bmatrix} 1 & 0 & -B_x^C \sin \hat{\theta}_B - B_y^C \cos \hat{\theta}_B \\ 0 & 1 & B_x^C \cos \hat{\theta}_B - B_y^C \sin \hat{\theta}_B \\ 0 & 0 & 1 \end{bmatrix}$$

Jacobian with Respect to Noise

$$W_k = \begin{bmatrix} \cos \hat{\theta}_B & -\sin \hat{\theta}_B & 0 \\ \sin \hat{\theta}_B & \cos \hat{\theta}_B & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Observation Model

$$H_k = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$V_k = \mathbf{I}_3$$

Velocity Model

State Vector

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{N}_k \\ \mathbf{V}_k \end{bmatrix}, \mathbf{N}_k = \begin{bmatrix} N_x^B \\ N_y^B \\ N_\theta^B \end{bmatrix}, \mathbf{V}_k = \begin{bmatrix} V_x^B \\ V_y^B \\ \omega^B \end{bmatrix}$$

Motion Model

$$\bar{\mathbf{x}}_k = \mathbf{x}_{k-1} \oplus \left(\mathbf{v}_{k-1} t + \frac{\mathbf{w}_k t^2}{2} \right)$$

$$\bar{\mathbf{v}}_k = \mathbf{v}_{k-1} + \mathbf{w}_k t$$

$$\bar{\mathbf{P}}_k = A_k \mathbf{P}_{k-1} A_k^T + W_k \mathbf{Q}_k W_k^T$$

Jacobian with Respect to State Vector

$$A_k = \begin{bmatrix} J_1 \oplus (\hat{\mathbf{x}}_{k-1}) & J_2 \oplus (\hat{\mathbf{x}}_{k-1}) \delta t \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_3 \end{bmatrix}$$

Jacobian with Respect to Noise

$$W_k = \begin{bmatrix} J_1 \oplus (\hat{\mathbf{x}}_{k-1}) \frac{\delta t^2}{2} \\ J_2 \delta t \end{bmatrix}$$

Observation Model

$$H_k = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$V_k = \mathbf{I}_3$$

Map Feature Compounding

This section describes the compounding operation to convert a 3D robot pose into a 2D feature representation.

Representation

- Pose:

$$\mathbf{N}_x^B = \begin{bmatrix} N_x^B \\ N_y^B \\ N_z^B \\ \psi_B \end{bmatrix} \sim \mathcal{N}(\hat{\mathbf{N}}_x^B, \mathbf{P}_B)$$

- Storage:

$$\mathbf{B}_x^F = \begin{bmatrix} B_x^F \\ B_y^F \end{bmatrix}$$

- Observation:

$$z_k = \mathbf{N}_x^F + \mathbf{B}_v^k, \quad \mathbf{B}_v^k \sim \mathcal{N}(0, \mathbf{P}_F)$$

Compounding Equation

The feature in 2D is compounded as:

$$\mathbf{N}_x^F = F(\mathbf{N}_x^B(\mathbf{B}_x^F + \mathbf{B}_v^k))$$

Where:

$$F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Jacobian Matrices

- Jacobian with Respect to the Pose (J_1):

$$J_1 = \frac{\partial(\mathbf{N}_x^B(\mathbf{B}_x^F + \mathbf{B}_v^k))}{\partial \mathbf{N}_x^B}$$

- Jacobian with Respect to the Feature (J_2):

$$J_2 = \frac{\partial(\mathbf{N}_x^B(\mathbf{B}_x^F + \mathbf{B}_v^k))}{\partial \mathbf{B}_x^F}$$

- **Jacobian with Respect to Noise (J_3):**

$$J_3 = \frac{\partial(\mathbf{N}_x^B(\mathbf{B}_x^F + \mathbf{B}_v^k))}{\partial \mathbf{B}_v^k}$$

Projection into 2D

The observation z_k in 2D space is:

$$z_k = F\mathbf{N}_x^F$$

Jacobian of Projection (J_{2D})

$$J_{2D} = \frac{\partial z_k}{\partial \mathbf{N}_x^F} = F$$

Map based EKF Localization

To implement map-based EKF localization, we extended the GFLocalization and MapFeature classes to add functionality for using feature-based maps. We then implemented the MBL_3DOFDDInputDisplacementMM_2DCartesianFeatureOM class, which integrates a 3DOF differential drive motion model and a 2D Cartesian feature map. For testing, we adjusted parameters like yaw_reading_frequency and xy_feature_reading_frequency to simulate scenarios with and without compass and feature updates. Data association was implemented using the ICNN algorithm, where we computed the innovation term, its covariance, and the Mahalanobis distance to match sensor measurements to map features.

4 Result and Discussion

The results from this lab are as follows;

I. Input Displacement Motion model with 2D Cartesian coordinate

a. Without Feature and compass update

The displacement motion model predicts the state based on the current state and control input. From the figure[1], it is observed that Model uncertainties lead to deviations, with small errors accumulating over time. Significant divergence occurs between predicted and actual states, increasing pose and feature position uncertainty.

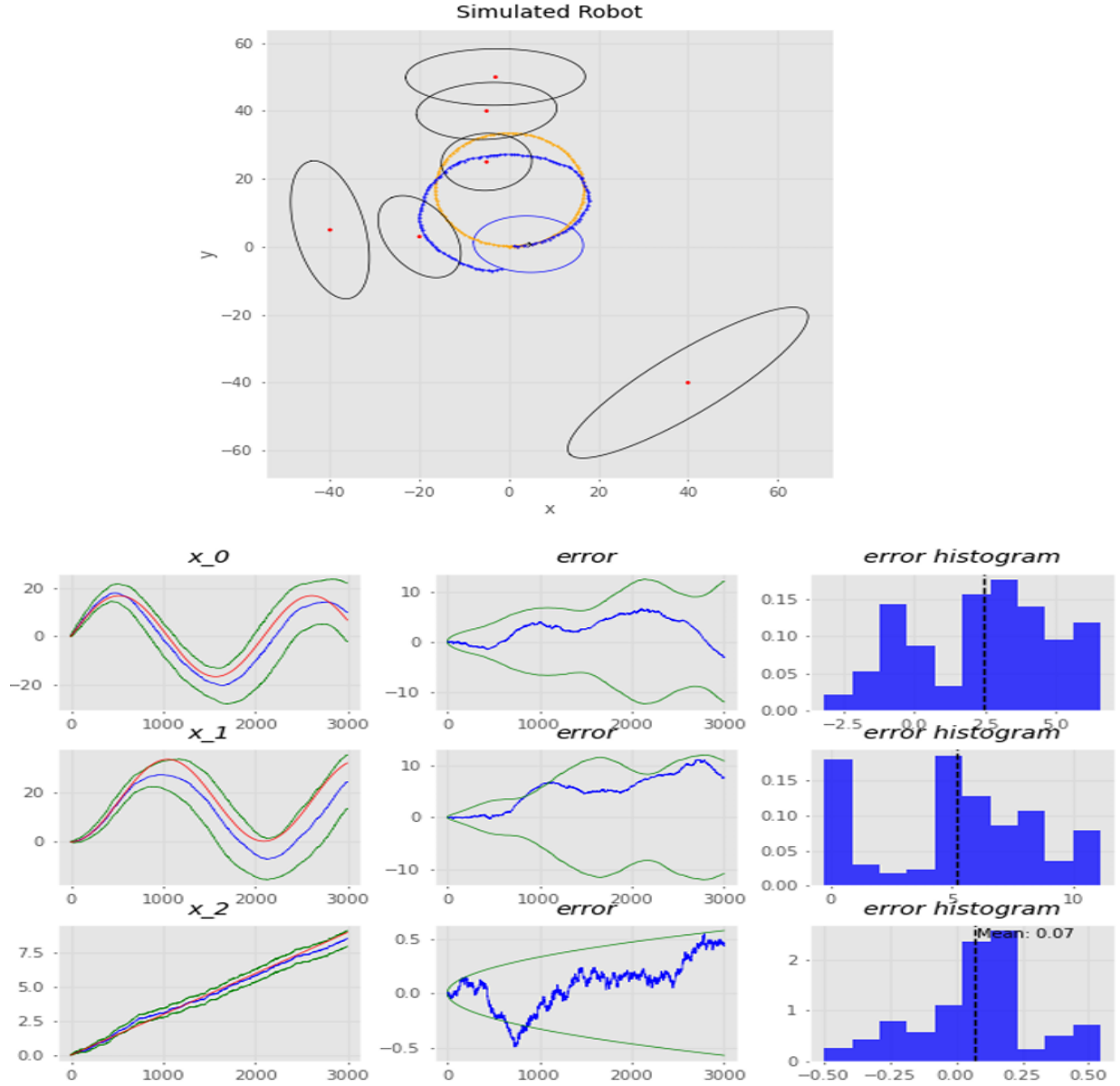


Figure 1: I a Input Displacement Motion model with 2D Cartesian coordinate Without Feature and compass update

b. With feature update at 50 step but without compass update

From the result figure [2], Features within the field of view and within a maximum range are considered valid. Feature updates every 50 steps allow correction of position and orientation. Drift in the system's state is reduced significantly.

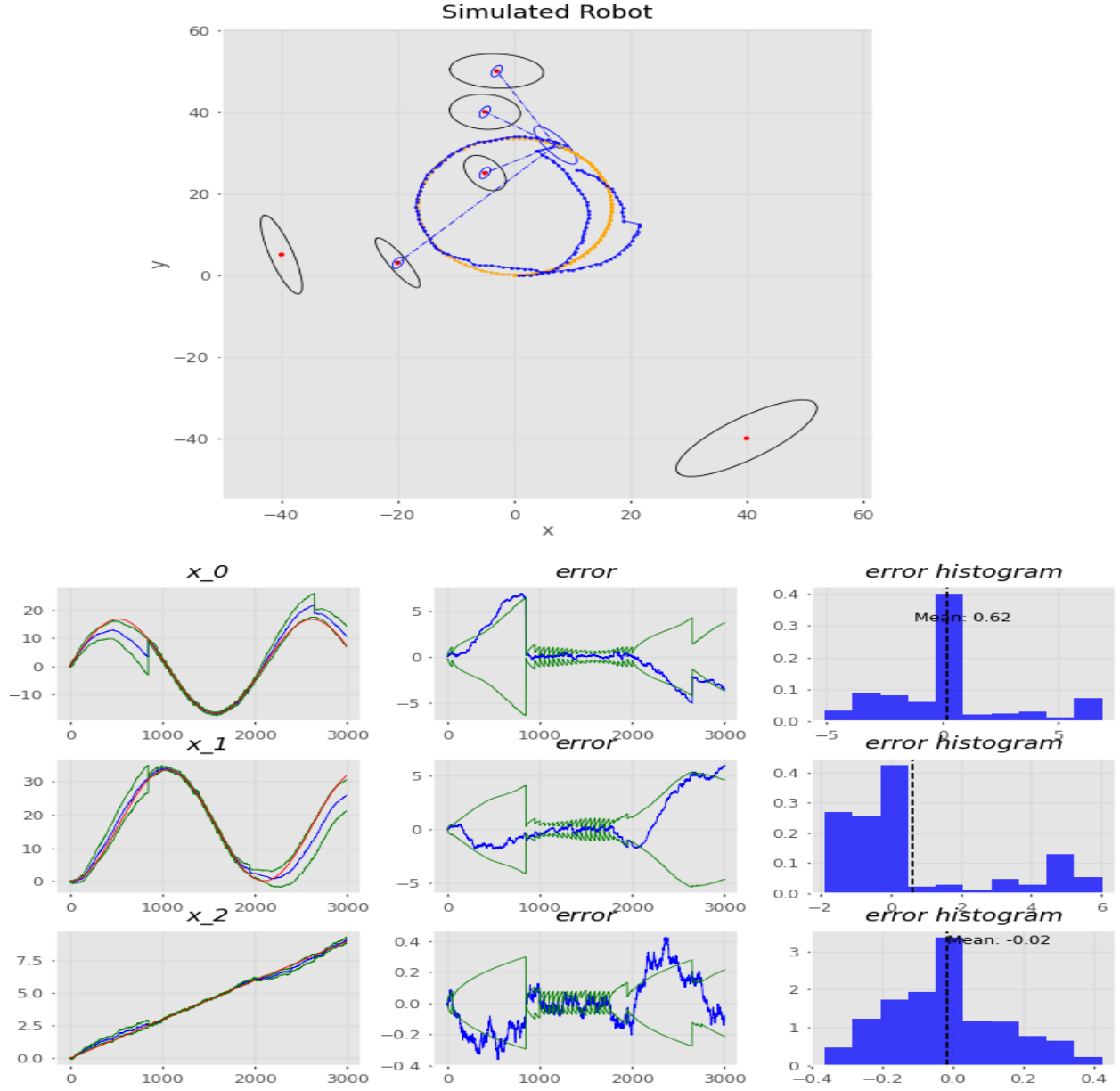


Figure 2: I b Input Displacement Motion model with 2D Cartesian coordinate With feature update at 50 step but without compass update

c. Both compass and feature update in 1 simulation step

From results figure [3], Continuous updates from compass and feature observations reduce accumulated errors. Compass updates refine orientation estimates and prevent drift. Frequent updates improve position and orientation accuracy.

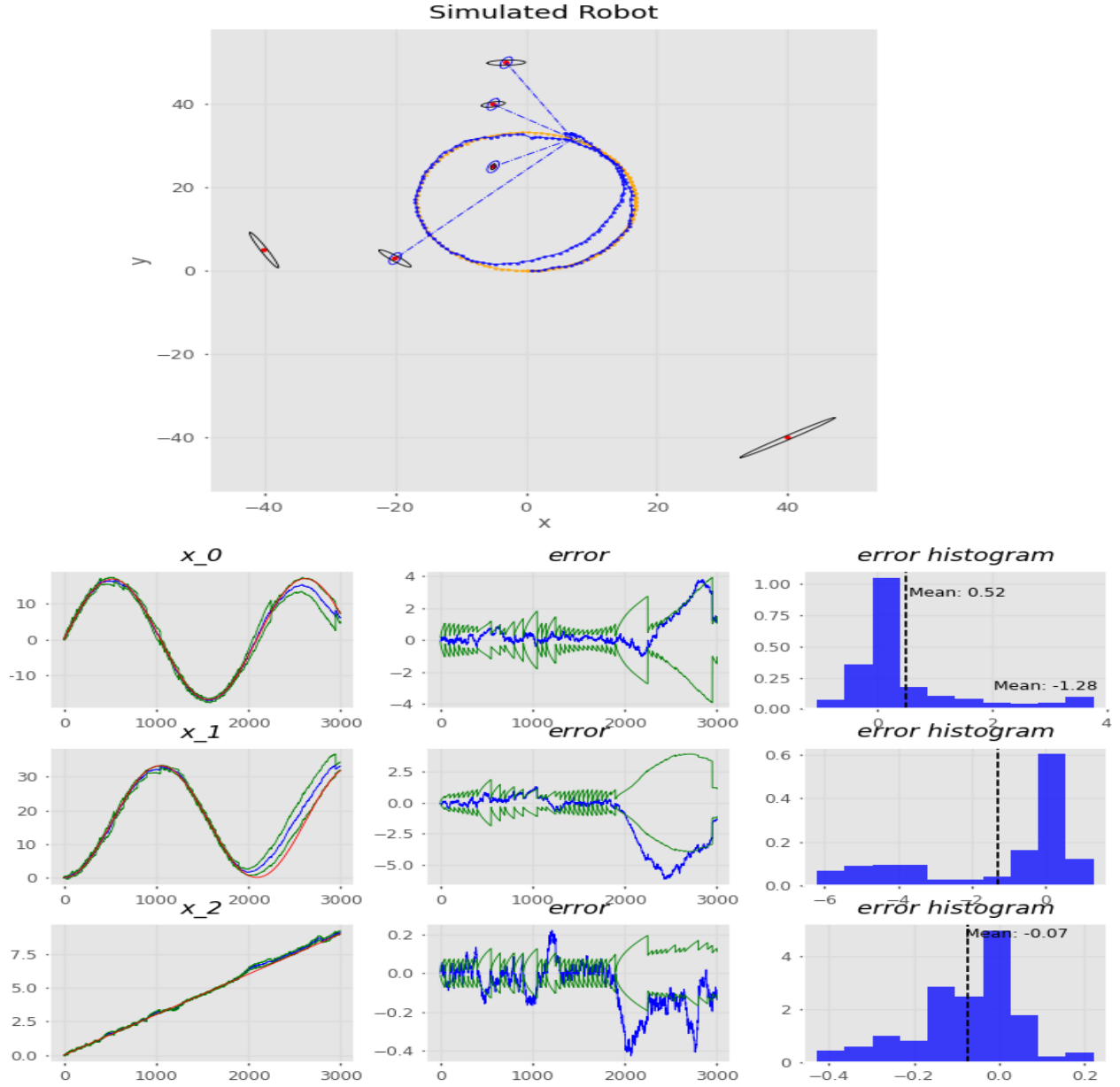


Figure 3: I c Input Displacement Motion model with 2D Cartesian coordinate Both compass and feature update in 1 simulation step

II. Input Displacement Motion model with stored in Cartesian but observed in Polar coordinate

a. Without Feature and compass update

In figure[4] Similar results to the Cartesian feature case: errors accumulate over time, leading to increased pose and feature position uncertainty.

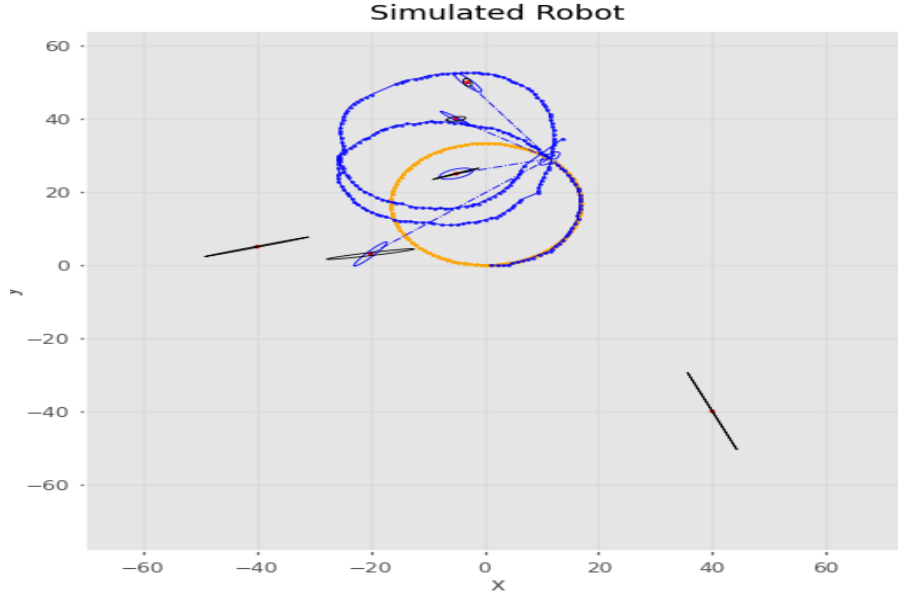


Figure 4: II a. Input Displacement Motion model with stored in Cartesian but observed in Polar coordinate Without Feature and compass update

b. With feature update at 50 step but without compass update

From figure [5] Feature updates every 50 steps reduce drift in position and orientation estimates. Corrections from polar feature observations improve system accuracy over time.

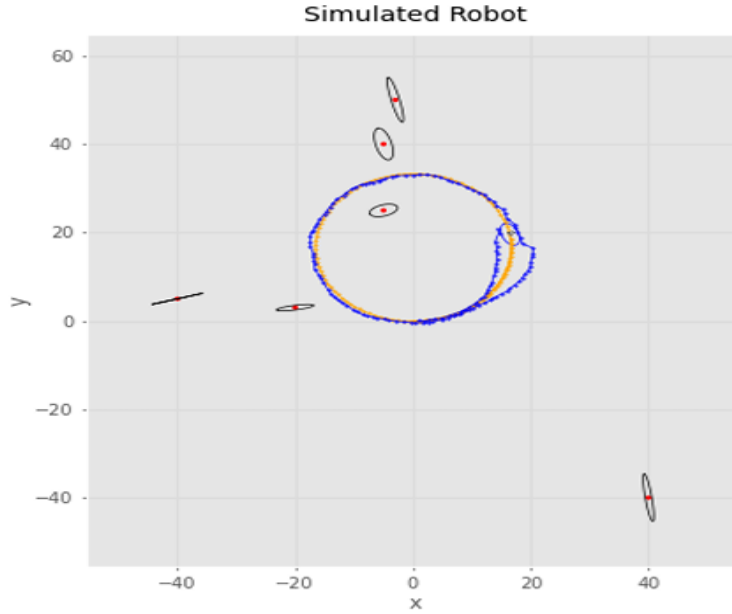


Figure 5: II b. Input Displacement Motion model with stored in Cartesian but observed in Polar coordinate With feature update at 50 step but without compass update

III. Constant velocity Motion model with 2D Cartesian coordinate

a. Without Feature and compass update

From figure [6] Acceleration noise in the constant velocity model causes errors in position estimates. Errors accumulate over time, leading to divergence between predicted and true positions. Pose and feature position uncertainty increases.

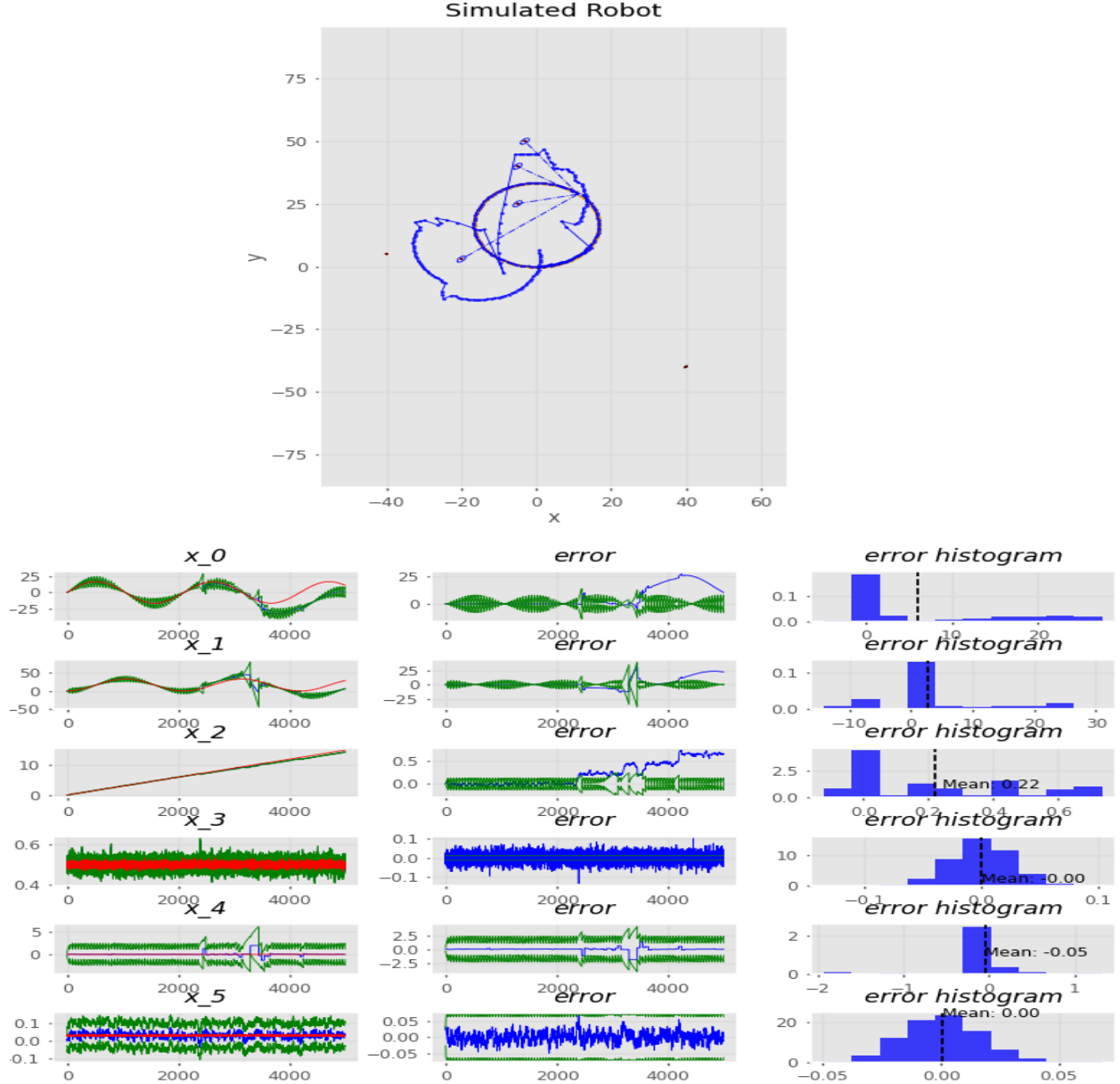


Figure 6: III a. Constant velocity Motion model with 2D Cartesian coordinate Without Feature and compass update

b. With feature update at 50 step but without compass update

From figure [7] Feature updates every 50 steps help correct the robot's estimated position and orientation. Drift is significantly reduced, resulting in improved localization accuracy.

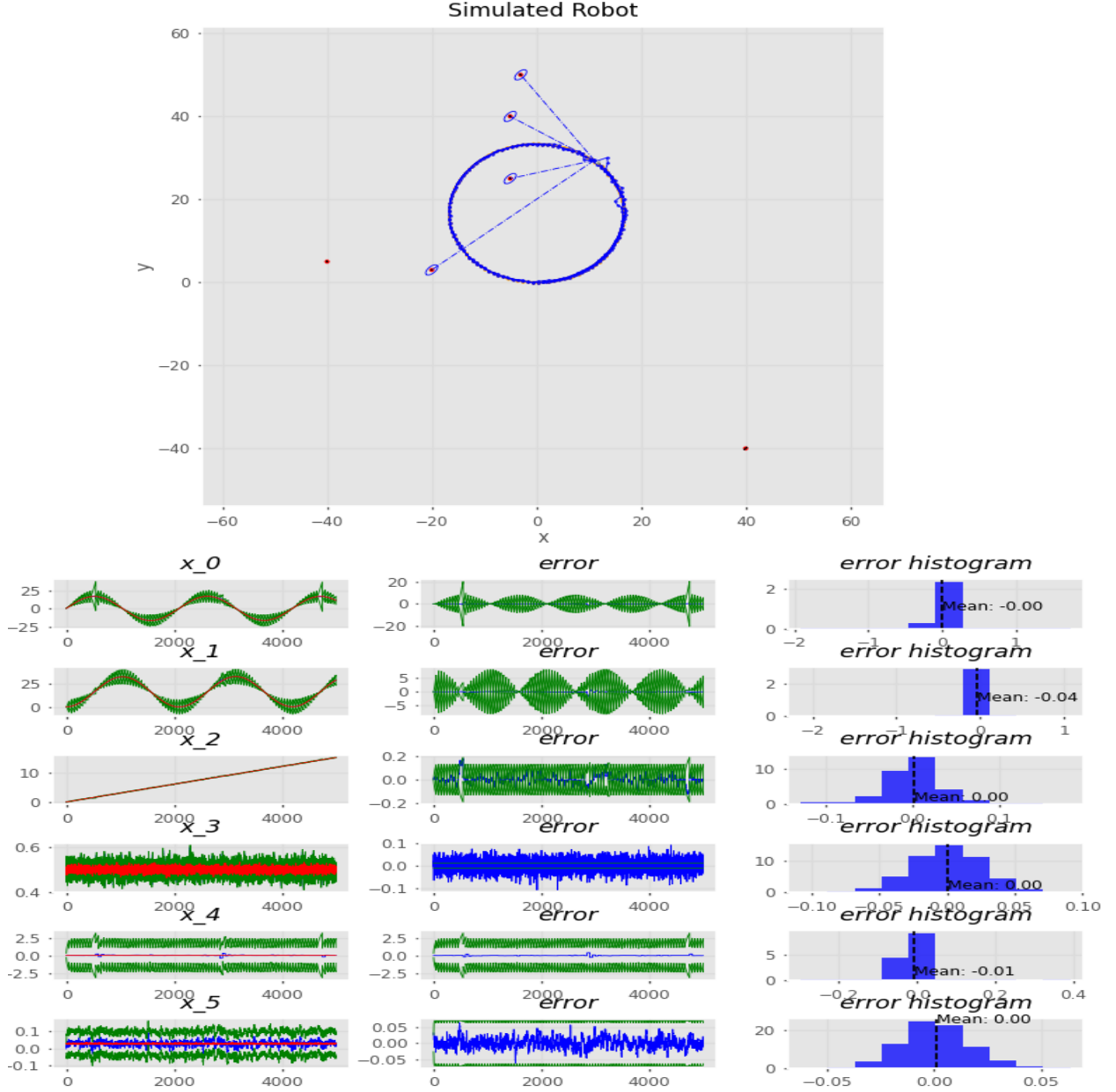


Figure 7: III b Input Displacement Motion model with 2D Cartesian coordinate With feature update at 50 step but without compass update

5 Problems

Challenges faced during implementation was plotting the Feature Observed and Stored in Polar result. we implemented the necessary code to run the MBL_3DOFDDInputDisplacementMM_2DPolarF

file, which facilitates map-based EKF localization for a differential drive mobile robot using input displacement and polar features. However, the plotting results doesnot seems accurate and valid as shown in figure[8].

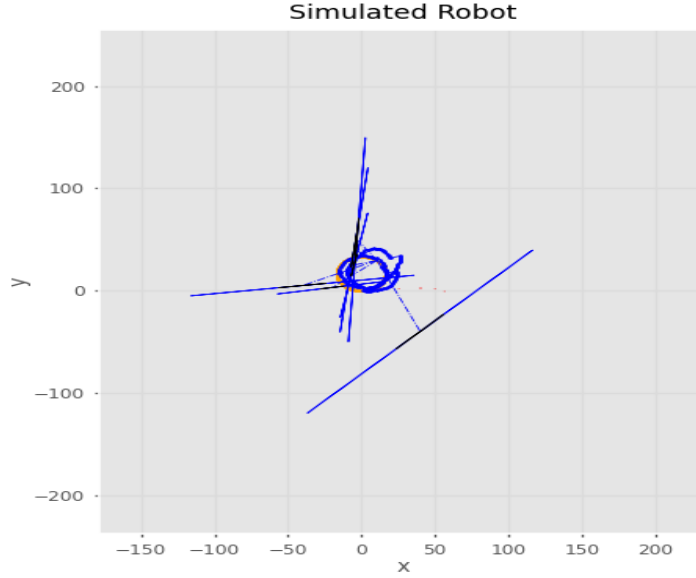


Figure 8: Error in plotting Feature Observed and Stored in Polar result

6 Conclusion

.The map-based EKF localization showed that frequent updates from the compass and features are crucial for improving accuracy and reducing drift in estimating the robot's position. Both the input displacement and constant velocity models worked well when updates were frequent, but without updates, errors and uncertainties built up over time. The results highlight how important it is to use feature observations within the robot's field of view to correct its position. However, there were challenges, like issues with plotting polar features, which need further investigation. Overall, this study demonstrates that EKF is a powerful tool for localization in nonlinear systems, but consistent updates are key to maintaining accuracy.